

Escuela Colombiana de Ingeniería Julio Garavito

Laboratorio 2

ARSW-2026-02-GR04

Alumnos:

- Esteban Valente Arenas
- Keyla Yunuette Serna Illescas

Agosto 18, 2026

SnakeRace-ARSW-Lab2

Reporte de Laboratorio 2 — Programación Concurrente (ARSW)

Escuela Colombiana de Ingeniería Julio Garavito — Arquitecturas de Software

Repositorio: SnakeRace-ARSW-Lab2

1. Introducción

Este laboratorio parte de una versión funcional pero **concurrentemente insegura** del juego SnakeRace: cada serpiente se mueve en su propio hilo, pero el código no protege correctamente el estado que esos hilos comparten entre sí y con la interfaz gráfica (Swing). El objetivo del laboratorio es identificar esos puntos de riesgo (condiciones de carrera, colecciones no seguras, esperas activas mal implementadas) y corregirlos aplicando el modelo de monitores de Java (`synchronized`, `wait()`, `notify()`/ `notifyAll()`), manteniendo el alcance de cada región crítica lo más pequeño posible.

El trabajo se divide en dos partes:

- **Parte I (calentamiento):** un ejercicio aislado y más simple — `PrimeFinder` — para practicar el patrón de suspensión/reanudación de hilos con `wait/notify` antes de aplicarlo al problema más grande.
- **Parte II (núcleo del laboratorio):** el análisis y corrección completa de SnakeRace, incluyendo una UI de Iniciar/Pausar/Reanudar con estadísticas consistentes.

2. Parte I — Calentamiento: control de hilos con wait/notify

2.1 Qué se implementó

`PrimeFinder.java` lanza **N hilos trabajadores** que buscan números primos indefinidamente, tomando cada uno el siguiente candidato de un contador compartido (`AtomicInteger`). Un hilo controlador (el `main`), cada `t` milisegundos, pausa a todos los trabajadores, espera a que confirmen que están detenidos, imprime cuántos primos se han encontrado, y bloquea esperando ENTER por consola antes de reanudar.

2.2 Diseño de sincronización

Qué lock se usa: un único monitor, la instancia de la clase interna `PauseGate` (concretamente su campo `lock`). Se decidió usar **un solo candado** para todo el estado relacionado con

pausa/reanudación (`paused`, `parked`, `totalWorkers`) en vez de varios candados, porque esas tres variables cambian juntas y de forma dependiente: separarlas en locks distintos habría obligado a coordinarlos entre sí, complicando el diseño sin ninguna ganancia de paralelismo real (la pausa es, por naturaleza, un punto de sincronización global).

Qué condición se espera:

- Cada trabajador espera `while (paused) lock.wait()`; dentro de `checkpoint()` — se llama una vez por iteración de su bucle de búsqueda.
- El controlador espera `while (parked < totalWorkers) lock.wait()`; dentro de `pauseAndAwaitAllParked()`.

Cómo se evitan los "lost wakeups": la regla aplicada es que la comprobación de la condición y la llamada a `wait()` nunca se separan: ambas ocurren dentro del mismo bloque `synchronized`, sobre el mismo lock que usa quien más tarde llama a `notifyAll()`. Si se comprobara la condición fuera del bloque sincronizado y solo se entraría al monitor para llamar a `wait()`, existiría una ventana de tiempo en la que otro hilo podría cambiar el estado y notificar *antes* de que el primer hilo alcanzara a bloquearse — esa notificación se perdería y el hilo quedaría esperando para siempre. Al mantener todo dentro de una única sección crítica esto es imposible: mientras un hilo tiene el lock para decidir si espera, ningún otro hilo puede tener el lock para notificar.

Adicionalmente se usa `while` (nunca `if`) para revisar la condición antes y después de cada `wait()`. Esto protege contra dos escenarios: los *spurious wakeups* que el JLS permite explícitamente (un hilo puede despertar sin que nadie haya llamado `notify`), y el caso en que `notifyAll()` despierta a varios hilos a la vez, pero la condición real de cada uno todavía no se cumple (por ejemplo, el controlador puede despertar por el aviso de un solo trabajador que recién se "parqueó", sin que el resto lo haya hecho todavía).

Ausencia de espera activa: en ningún punto hay sondeo (*polling*) de una bandera dentro de un bucle con `sleep` corto. Todo bloqueo real ocurre vía `wait()`, que libera la CPU por completo hasta ser notificado.

2.3 Cómo correr esta parte

```
javac PrimeFinder.java
java edu.eci.arsw.primefinder.PrimeFinder 4 3000
```

El primer argumento es el número de hilos trabajadores (por defecto 4), el segundo es el período de reporte en milisegundos (por defecto 3000). Cada `t ms` el programa se pausa solo, imprime el conteo de primos encontrados, y espera `ENTER` para continuar.

3. Parte II — SnakeRace concurrente

3.1 Cómo el código da autonomía a cada serpiente

Cada serpiente corre en su propio **hilo virtual** (`Executors.newVirtualThreadPerTaskExecutor()`), ejecutando una instancia independiente de `SnakeRunner`. Ese hilo decide, en su propio bucle y a su propio ritmo (`Thread.sleep` variable según si está en modo turbo), si gira aleatoriamente y cuándo invocar `Board.step(...)` para avanzar una celda. Usar hilos virtuales permite escalar a decenas o cientos de serpientes sin agotar los hilos del sistema operativo, ya que el costo de cada uno es muy bajo.

3.2 Condiciones de carrera identificadas

La más importante: `Snake.body` (`ArrayDeque<Position>`) se escribía desde el hilo de la serpiente (`advance()`, invocado dentro de `Board.step`) y se leía desde el hilo de Swing (EDT) al dibujar el tablero (`snapshot()`, `head()`), sin ningún candado. `ArrayDeque` no es thread-safe: este patrón puede producir `ConcurrentModificationException`, lecturas corruptas a mitad de una modificación, o publicación insegura de referencias — el riesgo es que la ventana del juego se congele con una excepción en cualquier momento, de forma no reproducible de manera determinista (justo lo que hace peligrosas a las condiciones de carrera).

Un segundo problema, más sutil, de **consistencia entre hilos**: el botón de pausa original solo llamaba a `clock.pause()`, que detiene el *ticker* de repintado. Cada `SnakeRunner` seguía su bucle de movimiento por completo ajeno a ese estado — las serpientes seguían avanzando con el juego "visualmente pausado". No es una condición de carrera en el sentido clásico (no hay corrupción de datos), pero es un defecto real: el estado mostrado al usuario no correspondía al estado real de la simulación, y volvía inútil cualquier estadística que se intentara leer durante la pausa.

3.3 Colecciones o estructuras no seguras en contexto concurrente

- `Snake.body` (`ArrayDeque`) — descrita arriba.
- `Board.mice` / `obstacles` / `turbo` (`HashSet`) y `Board.teleports` (`HashMap`) — estas ya estaban protegidas correctamente en el punto de partida: todo método que las toca es `synchronized` sobre `Board`, y los getters devuelven copias defensivas. Se mantuvo ese diseño porque ya era correcto.
- **Nueva estructura, `deathOrder`:** al agregar la mecánica de "muerte" de serpientes (necesaria para las estadísticas pedidas), varias serpientes pueden morir "al mismo tiempo" desde hilos distintos. Se eligió `ConcurrentLinkedQueue<Snake>` — una cola no

bloqueante — en lugar de una `ArrayList` sincronizada, porque el único uso real es agregar (`offer`) y leer el primero (`peek`), y así se evita introducir un candado adicional que compita con el de `Board`.

3.4 Esperas activas o sincronización innecesaria

No había bucles de sondeo explícitos (`while(!condicion){}` sin bloqueo), pero sí el problema funcionalmente equivalente descrito en 3.2: un mecanismo de "pausa" que no bloqueaba realmente a los hilos de movimiento. No se detectó tampoco sincronización *innecesaria* que se pudiera eliminar sin perder corrección: el único `synchronized` amplio (`Board.step`) se justifica en la sección 3.5.

3.5 Correcciones mínimas y regiones críticas — riesgo y solución

Cambio	Riesgo que resuelve	Cómo lo resuelve
Snake: sincronizar <code>advance()</code> , <code>snapshot()</code> , <code>head()</code> , <code>length()</code> , <code>occupies()</code> sobre el monitor de la propia instancia	Lectura/escritura concurrente del <code>ArrayDeque</code> del cuerpo (EDT vs. hilo de la serpiente)	Todo acceso al <code>Deque</code> pasa por una región crítica mínima: solo esas líneas, no el resto de la clase (dirección y vida siguen siendo <code>volatile</code> , sin lock, porque son lecturas/escrituras atómicas de una única referencia)
Nuevo <code>PauseController</code> (<code>wait/notify</code> , mismo patrón que la Parte I)	Pausa "falsa" que no detenía el movimiento real	Cada <code>SnakeRunner</code> llama a <code>awaitIfPaused()</code> una vez por iteración; si el juego está pausado, se bloquea de verdad con <code>wait()</code> hasta <code>resume()</code>
<code>PauseController.awaitAllParked(timeout)</code> usado antes de leer estadísticas	<i>Tearing</i> : leer el estado de una serpiente que todavía está a mitad de un	Bloquea al hilo que pide las estadísticas hasta que todos los <code>SnakeRunner</code> confirman (vía <code>wait/notify</code>) que

	step() cuando se pidió la pausa (la suspensión nunca es instantánea)	están parqueados; si se agota un timeout razonable, se muestran las estadísticas igual pero con una advertencia explícita, en vez de bloquear indefinidamente
Board.step(...) sigue siendo un único método synchronized, ahora también detecta colisión contra el cuerpo de cualquier serpiente	Dos serpientes podrían "pisar" la misma celda de forma inconsistente, o dos hilos podrían decidir la muerte de una tercera serpiente en instantes contradictorios	Al serializar el movimiento (una serpiente avanza a la vez dentro de este lock), la comprobación de colisión y el movimiento ocurren de forma atómica. Lo que no está dentro del lock: la decisión de girar al azar y el Thread.sleep de cada SnakeRunner, para no retener el lock más de lo necesario

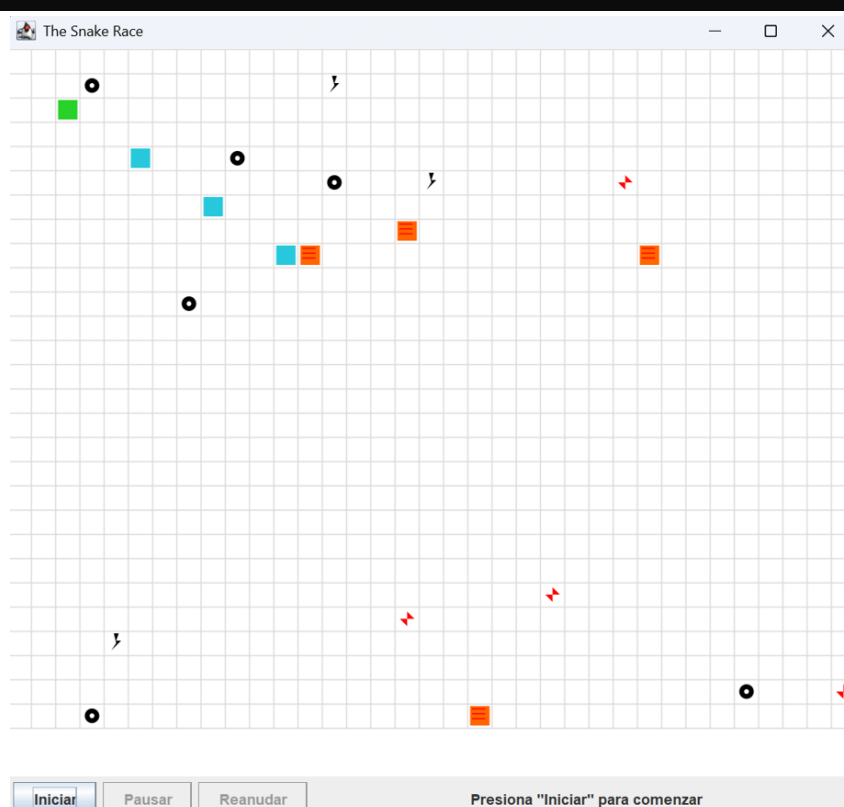
Justificación del alcance de Board.step: se evaluó partir este método en candados más finos, pero se descartó porque (a) el método ya es rápido — no hace I/O ni espera —, (b) solo se ejecuta una vez por iteración de cada SnakeRunner, nunca en un bucle ajustado, y (c) partir el lock habría obligado a tomar varios candados en un orden coordinado para verificar colisiones contra todas las demás serpientes, aumentando el riesgo de interbloqueo sin una ganancia de rendimiento medible a la escala de este laboratorio (hasta 40 serpientes probadas).

Por qué no hay riesgo de deadlock: dentro de Board.step (con el lock de Board ya tomado) se consulta other.occupies(...) de otras serpientes, lo cual toma internamente el lock de esa Snake. Como Board permite un único hilo dentro de step() a la vez, nunca hay dos hilos intentando adquirir el par de locks (Board, Snake) en órdenes distintos al mismo tiempo.

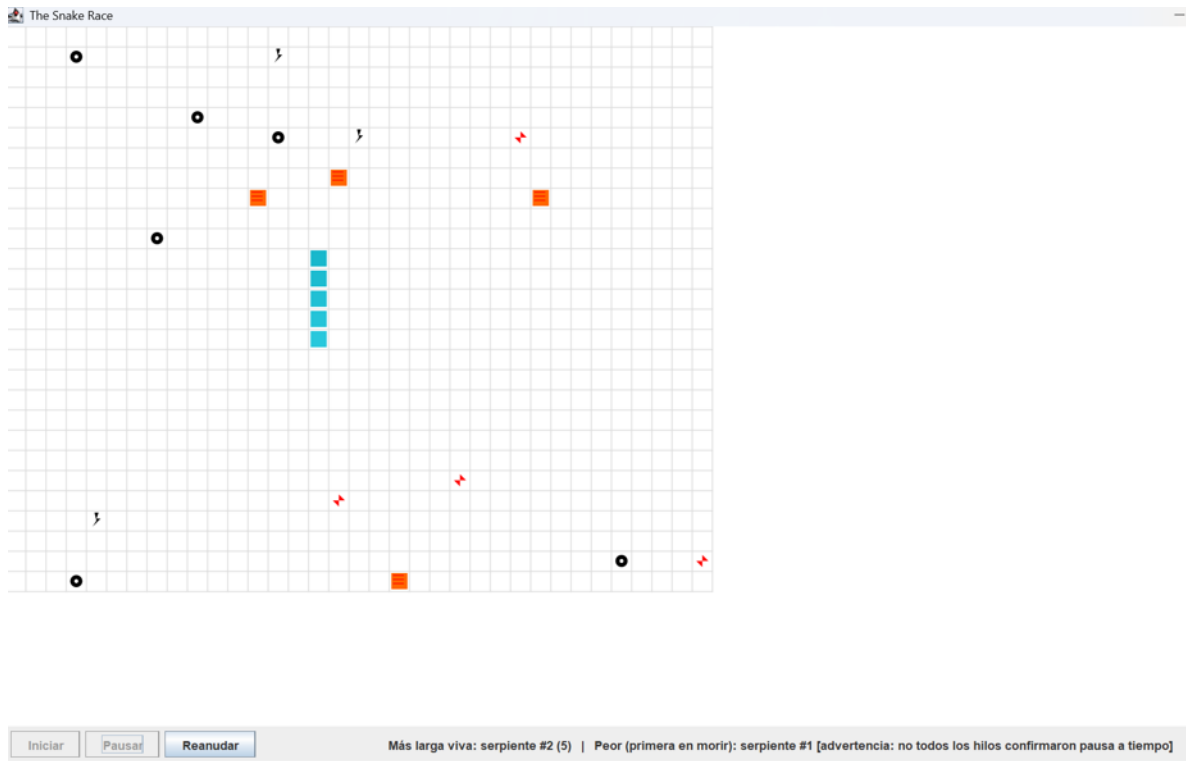
3.6 UI: Iniciar / Pausar / Reanudar con estadísticas

- El juego inicia automáticamente al abrir la ventana (`clock.start()`, hilos de `SnakeRunner` ya en ejecución).
- El botón alterna entre **Pausar** y **Reanudar**. Al pausar:
 - `pauseController.pause()` marca el estado global.
 - Un hilo virtual auxiliar llama a `awaitAllParked(500)` — sin bloquear el hilo de eventos de Swing, para no congelar la ventana.
 - Solo cuando retorna (todos parqueados, o venció el timeout) se calculan las estadísticas y se publican de vuelta al hilo de Swing con `invokeLater`.
- Estadísticas mostradas: **serpiente más larga viva** (recorriendo las vivas y comparando `length()`) y **peor serpiente** (`deathOrder.peek()`, la primera en morir).

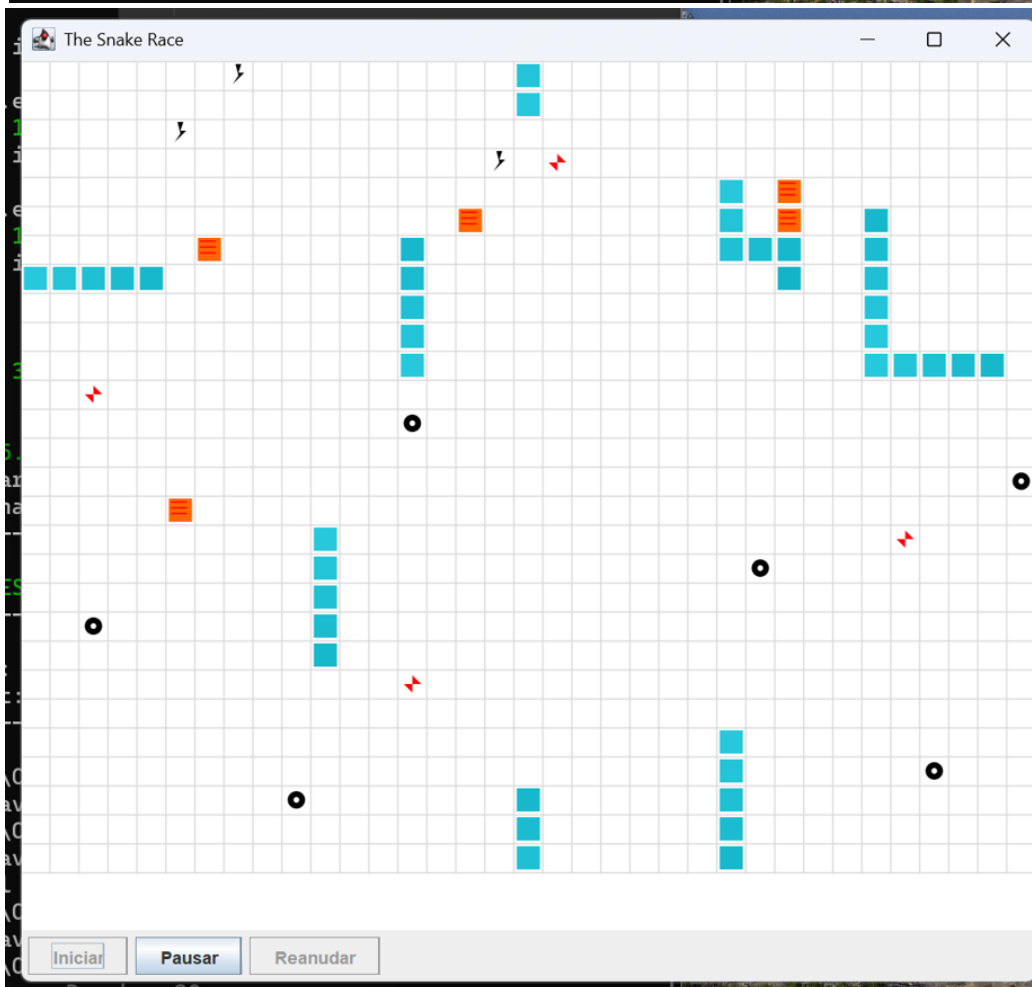
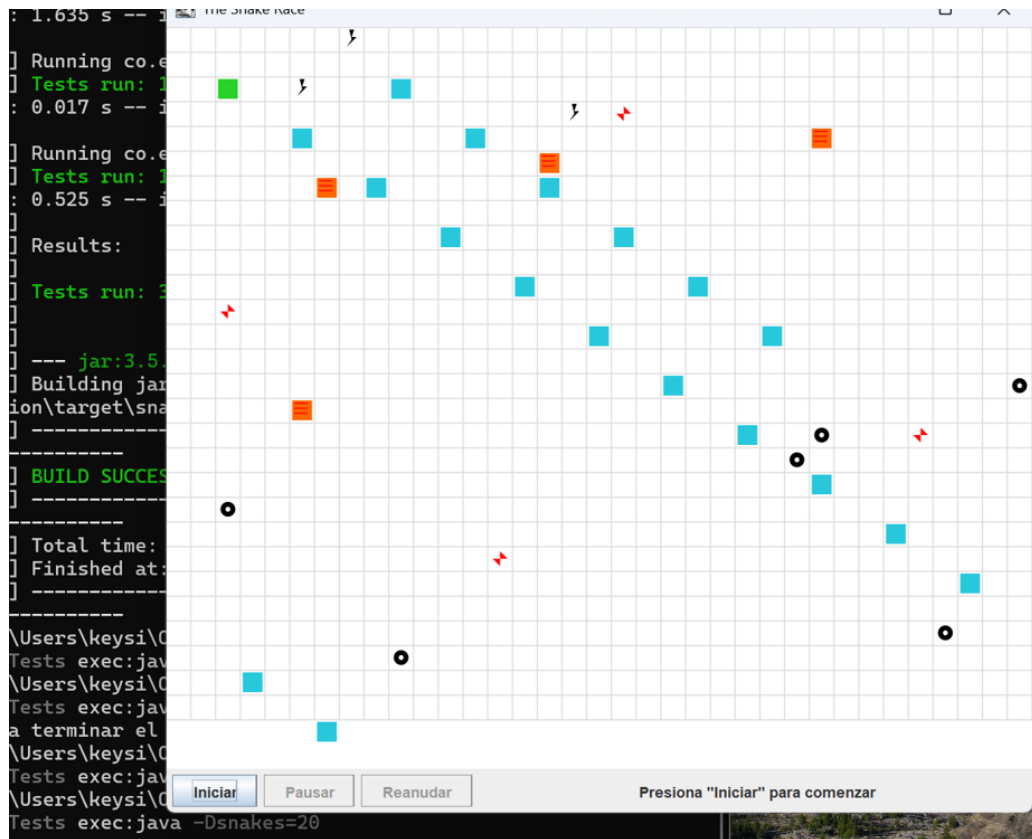
```
[INFO] -----  
-----  
PS C:\Users\keysi\OneDrive\Desktop\ARSW\snake-solution> mvn -q -  
DskipTests exec:java -Dsnakes=4
```

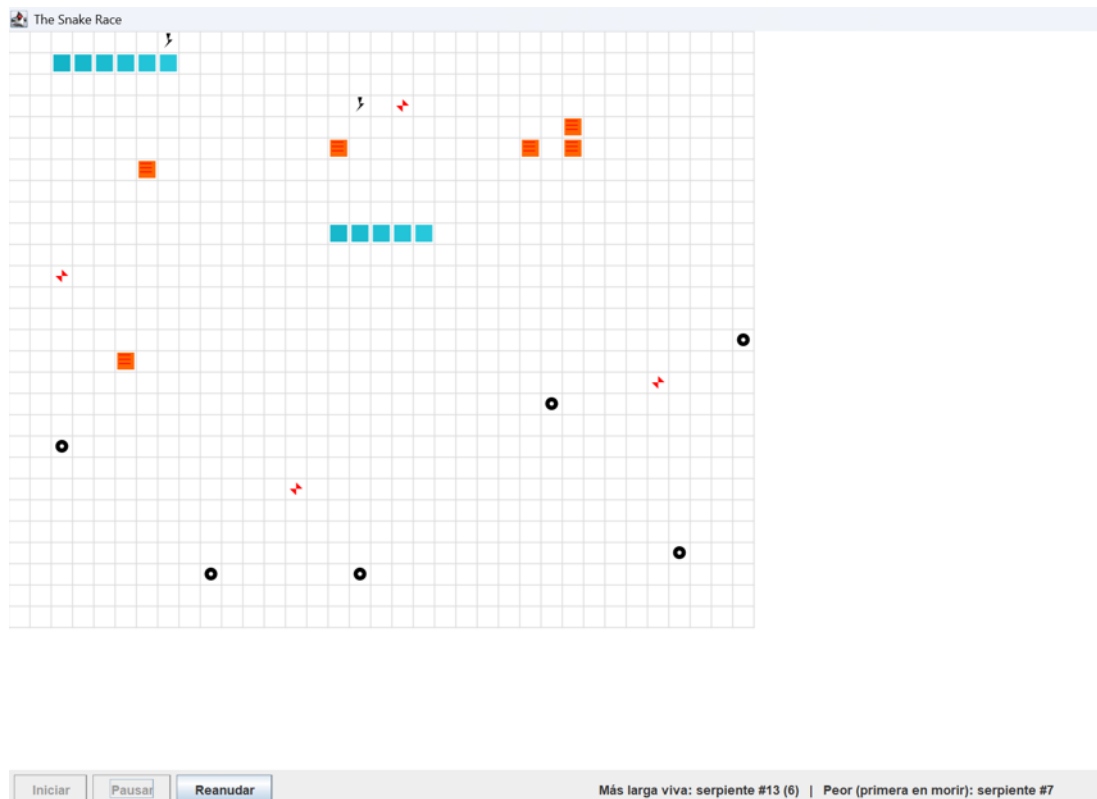






```
[INFO] -----  
PS C:\Users\keysi\OneDrive\Desktop\ARSW\snake-solution> mvn -q -  
DskipTests exec:java -Dsnakes=20|
```





4. Cómo compilar y correr las pruebas

Requisitos: **JDK 21** y **Maven 3.9+** instalados y en el PATH.

```
# Compila el proyecto y corre TODAS las pruebas automáticamente
mvn clean verify
```

```
[INFO]
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jar:3.5.0:jar (default-jar) @ snake-race-java21 ---
[INFO] Building jar: C:\Users\keysi\OneDrive\Desktop\ARSW\snake-
solution\target\snake-race-java21-1.2.0.jar
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 15.765 s
[INFO] Finished at: 2026-08-19T13:54:06-06:00
[INFO] -----
```

Este comando descarga las dependencias (JUnit 5), compila `src/main` y `src/test`, y ejecuta las tres clases de prueba. Debe terminar con `BUILD SUCCESS`. La salida detallada de cada prueba queda en `target/surefire-reports/`.

Para correr una sola clase de prueba (útil mientras se depura):

```
mvn test -Dtest=BoardConcurrencyTest
mvn test -Dtest=SnakeThreadSafetyTest
mvn test -Dtest=PauseControllerTest
```

Qué verifica cada una:

Prueba	Qué demuestra
SnakeThreadSafetyTest	Un hilo escribe (advance) 20 000 veces mientras otro lee (snapshot) simultáneamente sobre la misma serpiente — no debe lanzarse ninguna excepción.
BoardConcurrencyTest	30 serpientes en un tablero pequeño (12×12, colisiones frecuentes) corriendo en hilos virtuales — no debe lanzarse ninguna excepción de concurrencia y todos los hilos deben terminar limpiamente al ser interrumpidos.
PauseControllerTest	8 hilos se registran; el hilo principal pausa y espera a que todos confirmen (<code>awaitAllParked</code>); luego reanuda y verifica que todos se desbloqueen y terminen — prueba directa del contrato de <code>wait/notify</code> sin <i>lost wakeups</i> .

Para ejecutar el juego una vez compilado:

```
mvn -q -DskipTests exec:java -Dsnakes=4
mvn -q -DskipTests exec:java -Dsnakes=20
```

5. Conclusiones

El starter tenía dos defectos de concurrencia reales (acceso no sincronizado al cuerpo de la serpiente, y una pausa que no pausaba nada) y carecía de una mecánica de muerte necesaria para las estadísticas pedidas. Ambos se corrigieron aplicando el mismo patrón de monitor practicado en la Parte I (`synchronized` + `wait/notify`, condición revisada con `while`, comprobación y espera siempre atómicas respecto al mismo lock), manteniendo las regiones

críticas tan pequeñas como el problema lo permite. Las pruebas de concurrencia automatizadas y la prueba de carga manual con 40 serpientes confirman que el diseño se sostiene bajo estrés sin excepciones ni bloqueos.

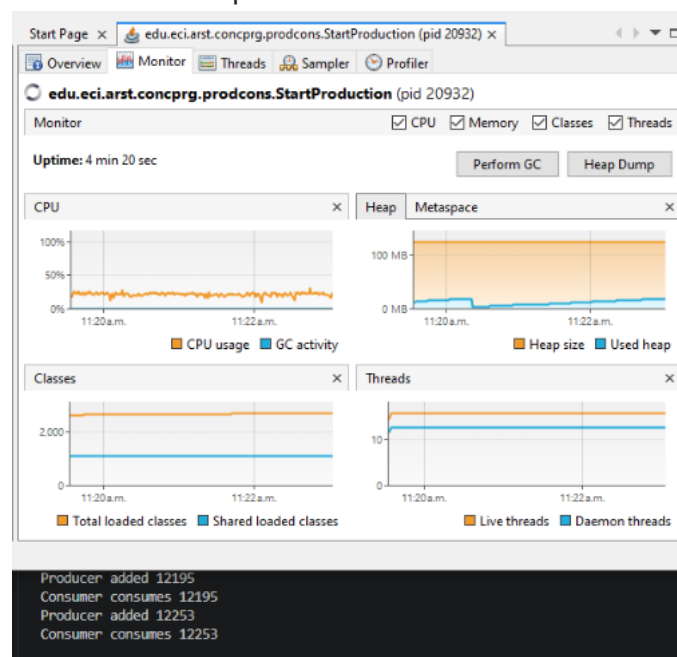
Parte 2

ConcurrentProgramming_Synchronization_DeadLocks_Ths Suspension

Parte I – Antes de terminar la clase.

Control de hilos con wait/notify. Productor/consumidor.

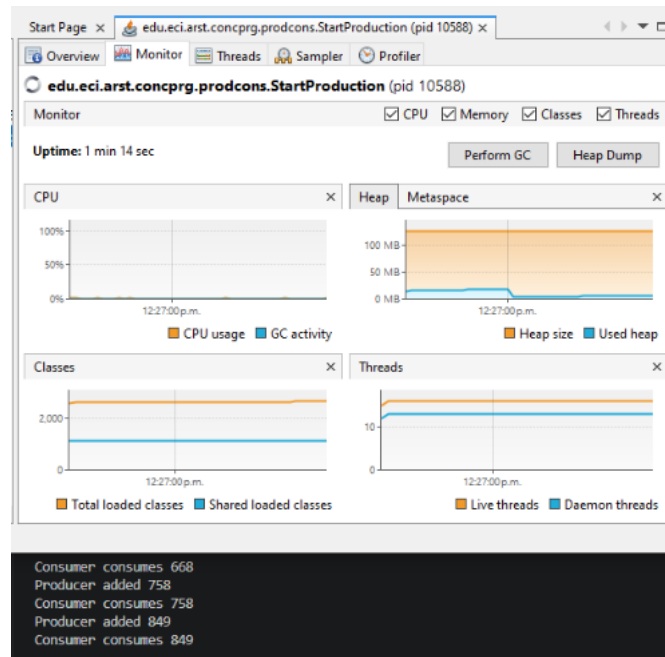
1. Revise el funcionamiento del programa y ejecútelo. Mientras esto ocurren, ejecute jVisualVM y revise el consumo de CPU del proceso correspondiente. A qué se debe este consumo?, cual es la clase responsable?



El problema que se presenta es debido a que dos hilos se están ejecutando simultáneamente, donde el hilo Consumidor pregunta cada segundo si hay algo por consumir, aunque no haya algo realmente lo que genera un uso excesivo en CPU, además que no existe una sincronización entre ambos hilos, por lo que tanto el

producto genera es consumido por el consumidor. La clase responsable es Consumer debido a que nunca termina su ciclo While.

- Haga los ajustes necesarios para que la solución use más eficientemente la CPU, teniendo en cuenta que -por ahora- la producción es lenta y el consumo es rápido. Verifique con JVisualVM que el consumo de CPU se reduzca.

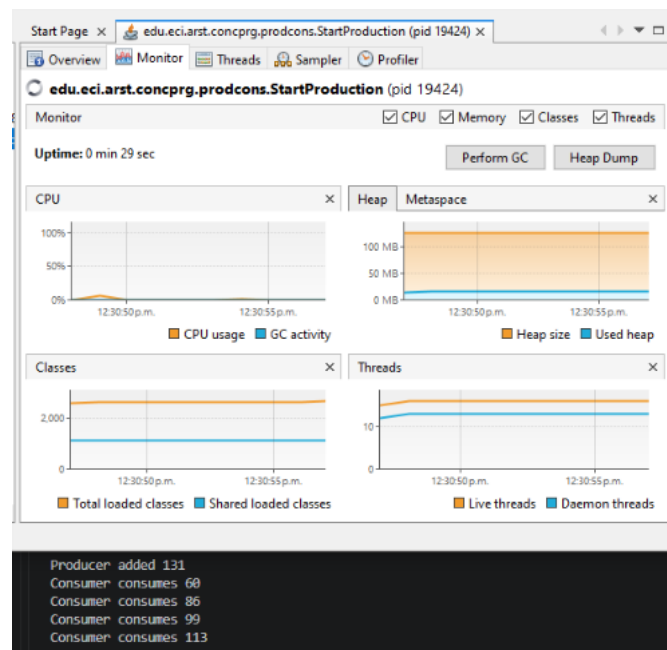


Lo que se hizo es sincronizar con el metodo synchronized, Estamos usando la cola como objeto de sincronización (ya que ahí están los elementos que "produce" y "consumen"), así el con ayuda de wait() el hilo Consumidor quede esperando, hasta que haya elementos a consumir y ya no consume CPU inútilmente.

Ahora el hilo Productor a través de un método sincronizado tiene que esperar Wait() cuando haya llegado al límite del stock. Además tiene que despertar (notifyAll()) a el hilo consumidor una vez que haya producido o haya algo que consumir y esperar 10 segundos para volver a producir. El Consumidor consume cada 1 segundo, pero cuando no hay espera y el Productor produce cada 5 segundos y despierta a el hilo Consumidor, cuando llega al límite stock espera.

- Haga que ahora el productor produzca muy rápido, y el consumidor consuma lento. Teniendo en cuenta que el productor conoce un límite de Stock (cuántos elementos debería tener, a lo sumo en la cola), haga que dicho límite se respete. Revise el API de la colección usada como cola para ver cómo garantizar que dicho límite no se

supere. Verifique que, al poner un límite pequeño para el 'stock', no haya consumo alto de CPU ni errores.



En base al código anterior, ahora se le invierte que el Producto produzca cada 1 segundo y el consumidor consuma cada 5 segundos, teniendo como stock 5 elementos.

Parte II. – Antes de terminar la clase.

Teniendo en cuenta los conceptos vistos de condición de carrera y sincronización, haga una nueva versión -más eficiente- del ejercicio anterior (el buscador de listas negras). En la versión actual, cada hilo se encarga de revisar el host en la totalidad del subconjunto de servidores que le corresponde, de manera que en conjunto se están explorando la totalidad de servidores. Teniendo esto en cuenta, haga que:

- La búsqueda distribuida se detenga (deje de buscar en las listas negras restantes) y retorne la respuesta apenas, en su conjunto, los hilos hayan detectado el número de ocurrencias requerido que determina si un host es confiable o no (`BLACK_LIST_ALARM_COUNT`).
- Lo anterior, garantizando que no se den condiciones de carrera.

```
Dirección IP a validar: 200.24.34.55
Número de hilos a usar: 4
ago 17, 2026 2:22:35 P. M. edu.eci.arst.blacklistvalidator.HostBlackListsValidator checkHost
INFORMACIÓN: Checked Black Lists:4,007 of 80,000
ago 17, 2026 2:22:35 P. M. edu.eci.arst.spamkeywordsdatasource.HostBlackListsDataSourceFacade reportAsNotTrustworthy
INFORMACIÓN: HOST 200.24.34.55 Reported as NOT trustworthy
The host was found in the following blacklists:[23, 50, 200, 500, 1000]
Tiempo de ejecución: 1791 ms
PS C:\Users\Esteban\Documents\Universidad\Escuela Colombiana de Ingenieros\Arquitectura de Software\1\LAB-02-02-Concurre
ntProgramming_Synchronization_DeadLocks_ThsSuspension>
```

Mediante de una variable atomica `AtomicInt`, la cual nos ayuda en acciones de lectura, escritura o modificación (como sumar o restar) se ejecutan de forma indivisible, sin que otro hilo pueda interrumpirlas a mitad del proceso, ya que si un hilo `BlackListSearchThread` encuentra una coincidencia numero `N` (definida en el codigo para que termine), el programa termine y detenga a todos los hilos que estaban esperando para seguir buscando. Cada hilo incrementa el contador mediante `incrementAndGet()` cuando encuentra una ocurrencia, para detener la busqueda se revisa cada lista, cada hilo consulta el contador compartido. Si ya alcanzó `BLACK_LIST_ALARM_COUNT`, termina su ejecución. Además, el hilo que alcanza el límite también termina inmediatamente. Y aqui se sigue usando el `join()` hace que el hilo principal espere hasta que cada hilo de búsqueda haya terminado antes de recolectar los resultados.

Parte III. – Avance para el martes, antes de clase.

Sincronización y Dead-Locks.



1. Revise el programa "highlander-simulator", dispuesto en el paquete `edu.eci.arsw.highlandersim`. Este es un juego en el que:
 - a. Se tienen `N` jugadores inmortales.
 - b. Cada jugador conoce a los `N-1` jugador restantes.
 - c. Cada jugador, permanentemente, ataca a algún otro inmortal. El que primero ataca le resta `M` puntos de vida a su contrincante, y aumenta en esta misma cantidad sus propios puntos de vida.
 - d. El juego podría nunca tener un único ganador. Lo más probable es que al final sólo queden dos, peleando indefinidamente quitando y sumando puntos de vida.
2. Revise el código e identifique cómo se implementó la funcionalidad antes indicada. Dada la intención del juego, un invariante debería ser que la sumatoria de los puntos de vida de todos los jugadores siempre sea el mismo (claro está, en un instante de

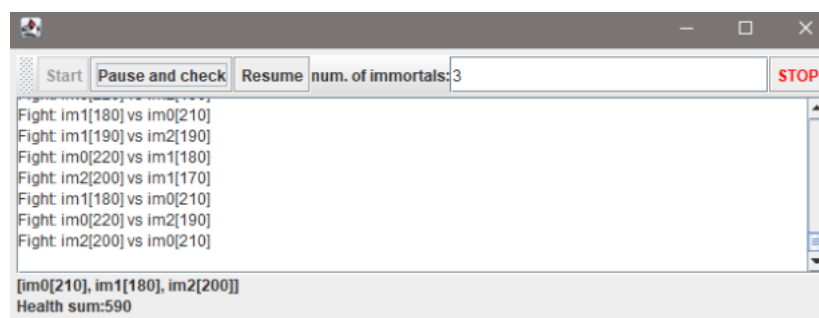
tiempo en el que no esté en proceso una operación de incremento/reducción de tiempo). Para este caso, para N jugadores, cual debería ser este valor?

$N \times 100$

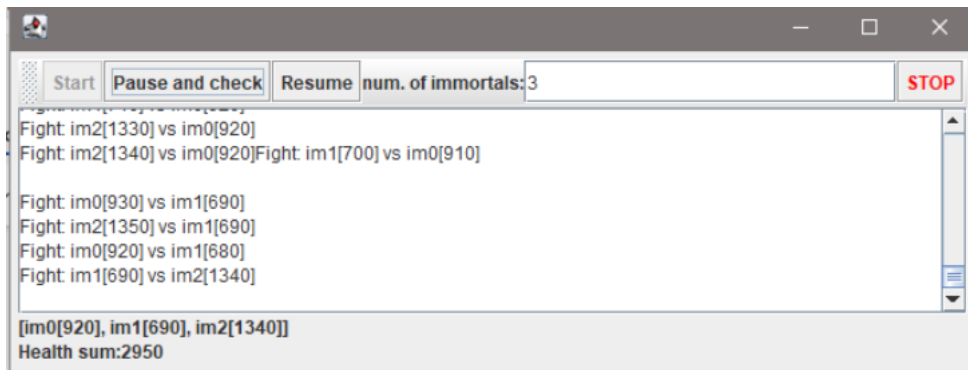
Porque cuando se crea un immortal este tiene una salud de 100, entonces si se crean N importales, se estan creando N veces salud de 100.

3. Ejecute la aplicación y verifique cómo funcionan las opción 'pause and check'. Se cumple el invariante?.

La opcion de "Pause and Check", solo nos da un resumen de la salud de cada inmortal y la suma de salud de todos ellos, pero los hilos siguen ejecutando, Lo que es la condición de carrera durante la lectura de la salud de cada inmortal, asi que con 3 hilos esta deberia ser 300 ya que entre ellos se están quitando la aumentando salud, y esta no debería de bajar ni subir de mas.



4. Una primera hipótesis para que se presente la condición de carrera para dicha función (pause and check), es que el programa consulta la lista cuyos valores va a imprimir, a la vez que otros hilos modifican sus valores. Para corregir esto, haga lo que sea necesario para que efectivamente, antes de imprimir los resultados actuales, se pausen todos los demás hilos. Adicionalmente, implemente la opción 'resume'.
5. Verifique nuevamente el funcionamiento (haga clic muchas veces en el botón). Se cumple o no el invariante?

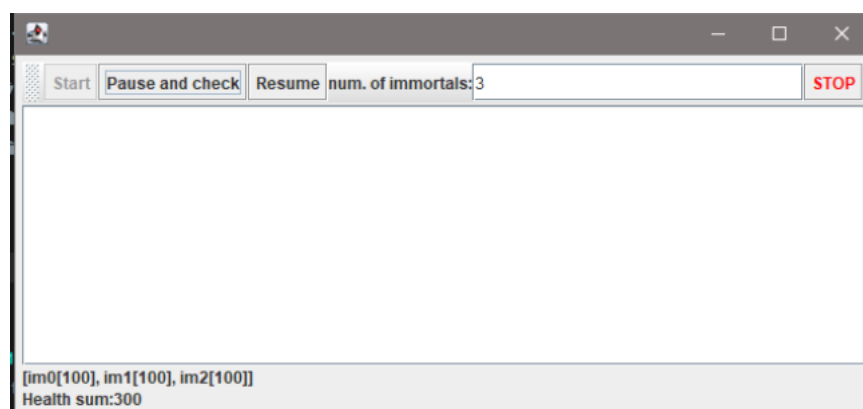


Después de implementar los botones correspondientes, no se cumple la invariante debido a que los inmortales siguen luchando, aunque ya estén muertos, por lo que causa un conflicto de condición carrera

6. Identifique posibles regiones críticas en lo que respecta a la pelea de los inmortales. Implemente una estrategia de bloqueo que evite las condiciones de carrera. Recuerde que si usted requiere usar dos o más 'locks' simultáneamente, puede usar bloques sincronizados anidados:

```
synchronized(locka){
    synchronized(lockb){
        ...
    }
}
```

7. Tras implementar su estrategia, ponga a correr su programa, y ponga atención a si éste se llega a detener. Si es así, use los programas jps y jstack para identificar por qué el programa se detuvo.



Después de la implementación de metodos sincronizados anidados, cayo en un deadlock por lo que ya no avanza y los hilos se quedan esperando a que uno a otro se desbloquee o libere el monitor.

```

PS C:\Users\Esteban> jps
11200 org.eclipse.equinox.launcher_1.7.200.v20260619-2039.jar
20736 Jps
17496 ControlFrame
23784
PS C:\Users\Esteban> jstack 17496

```

```

Found one Java-level deadlock:
*****
"im0":
  waiting to lock monitor 0x00000163fa3cf650 (object 0x000000008683a720, a edu.eci.arsw.highlandersim.Immortal),
  which is held by "im1"

"im1":
  waiting to lock monitor 0x00000163fb840760 (object 0x000000008683a828, a edu.eci.arsw.highlandersim.Immortal),
  which is held by "im2"

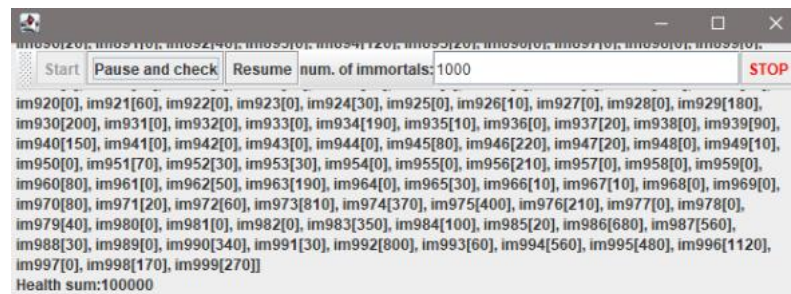
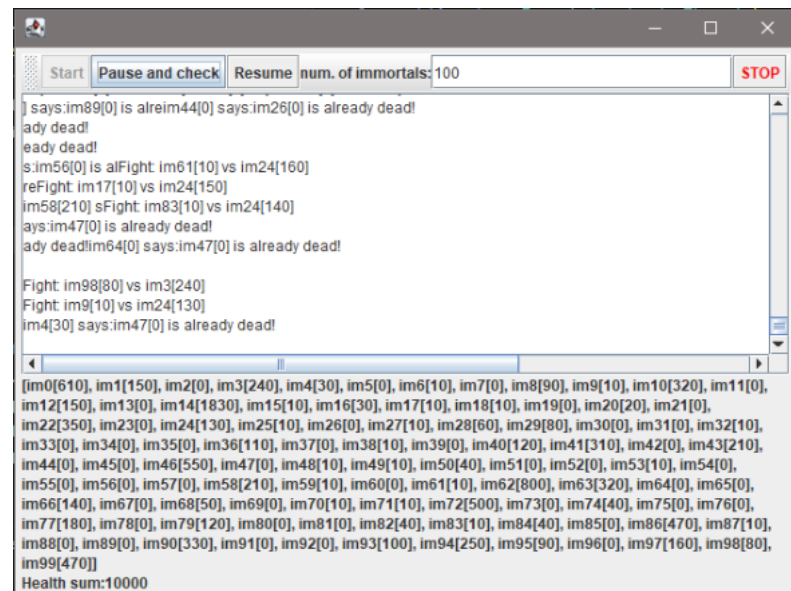
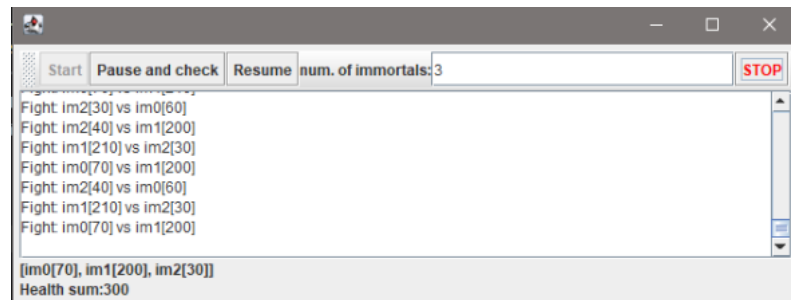
"im2":
  waiting to lock monitor 0x00000163fa3cf650 (object 0x000000008683a720, a edu.eci.arsw.highlandersim.Immortal),
  which is held by "im1"

Java stack information for the threads listed above:
*****
"im0":
  at edu.eci.arsw.highlandersim.Immortal.fight(Immortal.java:103)
  - waiting to lock <0x000000008683a720> (a edu.eci.arsw.highlandersim.Immortal)
  - locked <0x000000008683a688> (a edu.eci.arsw.highlandersim.Immortal)
  at edu.eci.arsw.highlandersim.Immortal.run(Immortal.java:87)
"im1":
  at edu.eci.arsw.highlandersim.Immortal.fight(Immortal.java:103)
  - waiting to lock <0x000000008683a828> (a edu.eci.arsw.highlandersim.Immortal)
  - locked <0x000000008683a720> (a edu.eci.arsw.highlandersim.Immortal)
  at edu.eci.arsw.highlandersim.Immortal.run(Immortal.java:87)
"im2":
  at edu.eci.arsw.highlandersim.Immortal.fight(Immortal.java:103)
  - waiting to lock <0x000000008683a720> (a edu.eci.arsw.highlandersim.Immortal)
  - locked <0x000000008683a828> (a edu.eci.arsw.highlandersim.Immortal)
  at edu.eci.arsw.highlandersim.Immortal.run(Immortal.java:87)
Found 1 deadlock.

```

Se identificó el proceso Java de la aplicación y posteriormente con `jstack` se confirmó la existencia de un deadlock. El reporte mostró que `im0`, `im1` e `im2` estaban esperando locks que estaban siendo retenidos por otros inmortales, generando una espera circular. Deadlock identificado con `jps`

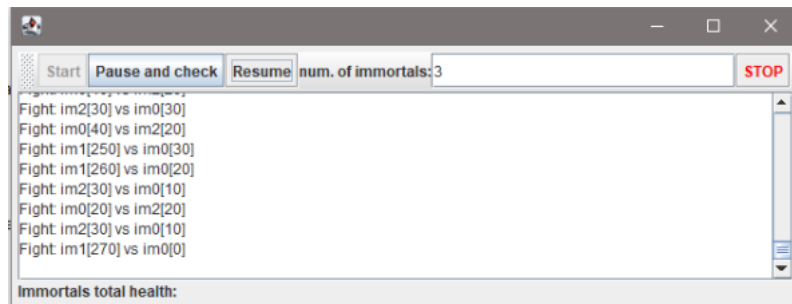
8. Plantee una estrategia para corregir el problema antes identificado (puede revisar de nuevo las páginas 206 y 207 de *Java Concurrency in Practice*).
9. Una vez corregido el problema, rectifique que el programa siga funcionando de manera consistente cuando se ejecutan 100, 1000 o 10000 inmortales. Si en estos casos grandes se empieza a incumplir de nuevo el invariante, debe analizar lo realizado en el paso 4.



Sin problemas se respeta la invariante despues de la implementación de ordenar los locks, o como es que los hilos deben de acceder aplicando la regla de que el menor hilo debe ir primero/bloquear primero

10. Un elemento molesto para la simulación es que en cierto punto de la misma hay pocos 'inmortales' vivos realizando peleas fallidas con 'inmortales' ya muertos. Es necesario ir suprimiendo los inmortales muertos de la simulación a medida que van muriendo. Para esto:

- a. Analizando el esquema de funcionamiento de la simulación, esto podría crear una condición de carrera? Implemente la funcionalidad, ejecute la simulación y observe qué problema se presenta cuando hay muchos 'inmortales' en la misma. Escriba sus conclusiones al respecto en el archivo RESPUESTAS.txt.
Después de implementar que se remuevan los inmortales muertos de la lista, cuando quedan dos luchando y uno muere da un Exception ya que el Bound debe ser positivo es decir debe haber un contrincante. A la vez de generar una condición carrera debido a que múltiples hilos acceden a la lista pero puede que mientras pasa eso un hilo es eliminado y este iba a hacer accedido por un hilo dando la condición carrera.
- b. Corrija el problema anterior **SIN hacer uso de sincronización**, pues volver secuencial el acceso a la lista compartida de inmortales haría extremadamente lenta la simulación.



Cambiando la lista por una estructura concurrente como CopyOnWriteArrayList, evitando bloquear explícitamente la población con synchronized. Esto permite que los hilos continúen ejecutando peleas concurrentemente mientras los inmortales muertos son retirados sin que sean accedidos mientras.

11. Para finalizar, implemente la opción STOP.

